

Trabajo Práctico 1 — Conceptos de Seguridad

Auditoría y Seguridad de Sistemas · ARyS · IF046

UNPSJB — Sede Trelew

Soler, Franco Martín

Parte A — El inventario y la tríada

A.1 / A.2 — Activos y pilar CIA crítico

Activo	Pilar CIA crítico	Justificación
Clave privada (.pem) de acceso a la VM de producción	Confidencialidad	Es la llave de entrada a producción. Si se filtra, un tercero entra con mis mismos permisos. Perderla es recuperable rotando el par de claves; que se filtre, no.
Clave SSH de acceso a los repositorios (frontend y backend)	Confidencialidad	Si se filtra, el atacante no solo lee el código: puede modificarlo y subirlo. La pérdida de confidencialidad de la clave se convierte en pérdida de integridad del software.
Copias locales de bases de datos de producción	Confidencialidad	Contienen datos reales de producción fuera del entorno controlado que los protege. Es el activo de mayor exposición: si la notebook se pierde o es comprometida, el daño ya está hecho.
Esquema de usuarios y permisos de la base de datos de producción	Integridad	Define quién puede leer y quién puede escribir en producción. Si alguien lo altera y se otorga permisos de escritura, el control de acceso deja de existir sin que nadie se entere. Lo crítico no es que se vea, es que no se modifique.
La VM de producción y los servicios que expone	Disponibilidad	Si la VM se cae, los sistemas dejan de operar. Acá el pilar crítico se invierte: importa menos quién ve el contenido que el servicio siga en pie.

A.3 — Análisis de riesgo del activo principal

Campo	Valor
Activo	Copias locales de bases de datos de producción
Amenaza	Acceso físico no autorizado al equipo: robo o extravío de la notebook, o un tercero que aprovecha una sesión desatendida sin bloquear.
Vulnerabilidad	Las copias se guardan sin cifrar en el disco. La contraseña del sistema operativo protege la sesión, no el archivo: alcanza con arrancar desde otro medio o extraer el disco para leerlo en texto plano.
Impacto	Alto
Probabilidad	Media

El impacto es Alto porque las copias contienen datos personales identificables junto con género y orientación política. La Ley 25.326 de Protección de Datos Personales clasifica las opiniones políticas como dato sensible, así que una filtración no solo perjudica a las personas listadas: expone al organismo a responsabilidad legal. Además el daño es irreversible — una clave comprometida se rota, unos datos filtrados no vuelven.

La probabilidad es Media porque habitualmente bloqueo el equipo, lo que reduce la ventana de una sesión desatendida. Pero el bloqueo no cubre el escenario de robo o extravío: sin cifrado de disco, cualquiera con el equipo en la mano llega al archivo.

Parte B — La tríada en acción (laboratorio)

Todos los comandos se ejecutaron dentro del contenedor Docker del repositorio (imagen arys-lab:bookworm, Debian bookworm-slim), lo que fija las versiones de las herramientas y aísla el laboratorio del sistema anfitrión. Los bloques monoespaciados reproducen la transcripción literal de la terminal.

B.1 — Integridad: funciones de hash

Comandos ejecutados dentro del contenedor arys-lab:bookworm (docker run --rm -it -v "\$PWD/tp/tp1/work:/lab" arys-lab:bookworm).

```
$ echo "Transferir 1000 a la cuenta 55" > orden.txt
sha256sum orden.txt
0e1280abab19bbd9cddaedf53cce80db1e7c889c46236115886c12366dcf5859  orden.txt

$ echo "Transferir 9000 a la cuenta 55" > orden.txt
sha256sum orden.txt
7d943b825c066b4c87ddf9e833d3a206d449beee071b6c3a4d2941239a5b9e5f  orden.txt
```

Respuesta

Cambiar un solo dígito (1000 -> 9000) produjo un hash completamente distinto: no comparten ni un tramo reconocible, aunque el contenido del archivo casi no cambió.

Esto es el efecto avalancha del hash: una diferencia mínima en la entrada se traduce en una salida totalmente distinta e impredecible, nunca en un hash "parecido".

Esto es lo que permite detectar una violación de integridad: si guardo el hash del archivo cuando confío en su contenido, y más tarde el hash recalculado no coincide, sé que el archivo cambió, sin tener que comparar el contenido byte a byte ni saber qué se modificó.

Pero el hash no sirve para deshacer el cambio. Es una función de una sola vía: a partir del hash no se puede reconstruir el contenido original. El hash me avisa que "orden.txt" fue alterado, pero no me devuelve la orden original de 1000 ni me dice qué se modificó. Para eso hace falta otra cosa (un backup, un control de versiones), no el hash.

B.2 — Confidencialidad: cifrado simétrico

gpg -c requiere un TTY interactivo para el prompt de contraseña (pinentry), que no se renderizaba bien en la terminal usada; se ejecutó el mismo comando en modo no interactivo (--pinentry-mode loopback) para obtener una transcripción reproducible. El efecto es idéntico al de la consigna: cifrado simétrico con contraseña.

Cifrado

```
$ gpg -c orden.txt
-rw-r--r-- 1 root root 31 orden.txt
-rw-r--r-- 1 root root 110 orden.txt.gpg
```

Contenido cifrado: binario ilegible

```
$ file orden.txt.gpg && cat orden.txt.gpg
orden.txt.gpg: GPG symmetrically encrypted data (AES256 cipher)
<bytes binarios no imprimibles -- contenido cifrado, no reproducible como texto>
```

Descifrado

```
$ gpg -d orden.txt.gpg
gpg: AES256.CFB encrypted data
Transferir 9000 a la cuenta 55
gpg: encrypted with 1 passphrase
```

Respuesta

El pilar que protege el cifrado es la confidencialidad: que solo quien tiene la contraseña pueda leer el contenido. Se comprobó arriba con "file" y "cat": sin la clave, el archivo es indistinguible de ruido.

Si alguien roba orden.txt.gpg pero no la contraseña, la confidencialidad no se viola: el cifrado sigue haciendo su trabajo aunque el archivo ya no esté solo en mi poder. Tener el archivo en la mano no sirve de nada sin la clave.

Si además borra mi única copia, ahí sí pierdo algo, pero es disponibilidad, no integridad: el archivo no fue alterado, simplemente dejó de estar accesible. Integridad hubiera fallado si alguien modificaba el contenido cifrado o el texto original; acá el dato desaparece intacto, no corrupto.

B.3 — Confidencialidad e integridad: control de acceso

```
$ ls -l orden.txt
chmod 600 orden.txt
ls -l orden.txt
-rw-r--r-- 1 root root 31 orden.txt
-rw----- 1 root root 31 orden.txt
```

Respuesta

chmod 600 son tres dígitos octales, uno por cada audiencia del archivo: dueño, grupo y otros. El 6 (4+2 = lectura+escritura) es para el dueño; los dos 0 restantes son para el grupo y para "otros", y significan que esas dos audiencias no tienen ningún permiso.

Esto conecta con dos pilares a la vez: confidencialidad, porque un usuario del sistema que no sea el dueño no puede leer el contenido; e integridad, porque tampoco puede escribirlo ni modificarlo.

Con el principio de menor privilegio la relación es directa: un archivo tiene tres audiencias posibles (dueño, grupo, otros), y 600 le da acceso a exactamente una, la única que efectivamente necesita leer y escribir ese archivo. Ni el grupo ni "otros" tienen una razón legítima para tocarlo, así que no reciben ningún permiso: es el mínimo suficiente para que el trabajo se haga, ni un bit más.

La excepción es root, y ya se vio en el ejercicio B.3 anterior: el usuario lab recibió "Permission denied" al leer un archivo 600, pero root leyó el mismo archivo sin problema. Esto es porque root no está sujeto a la verificación de permisos de archivos: el kernel directamente no hace esa comprobación cuando el proceso corre como root. chmod protege contra otros usuarios normales del sistema, no contra quien lo administra.

B.4 — Disponibilidad: el backup que se prueba

```
$ cp orden.txt orden.bak
rm orden.txt
cp orden.bak orden.txt
cat orden.txt
Transferir 9000 a la cuenta 55
```

Respuesta

El pilar que protege el backup es la disponibilidad: si algo le pasa al archivo original, en cualquier momento se puede restaurar desde la copia y seguir teniendo el dato accesible.

Un backup que nunca se probó restaurar no sirve como control real porque no está garantizado que efectivamente sostenga la disponibilidad: puede estar mal configurado, contener datos corruptos, o fallar el proceso de restauración, y no hay forma de saberlo hasta que se necesita usarlo. Para entonces ya es tarde.

Parte C — Del riesgo al control

Control	Tipo	Pilar CIA	Fundamento
Cifrar las copias locales de bases de datos (o el disco completo)	Preventivo	Confidencialidad	Ataca directamente la vulnerabilidad identificada (copias sin cifrar en disco): si roban el equipo, el contenido queda ilegible sin la passphrase, tal como se demostró en el ejercicio B.2.
Registro de auditoría sobre accesos al archivo	Detectivo	Confidencialidad	No evita el acceso no autorizado, pero permite detectar quién abrió o leyó el archivo, lo que da alerta temprana ante un acceso indebido.

Si solo pudiera aplicar un control, elegiría el cifrado: ataca directamente la vulnerabilidad identificada en A.3 y reduce el riesgo de confidencialidad a prácticamente cero, sin importar si el equipo termina en manos de un tercero. El costo real no es "perder la posibilidad de recuperar el dato" -- eso sería el costo de no tener backup -- sino depender de la passphrase: si la olvido o la pierdo, soy yo quien se queda sin poder abrir mis propias copias, es decir, el riesgo de confidencialidad se traslada a un riesgo de disponibilidad. Aun así, es un costo manejable (una passphrase guardada con cuidado) frente al daño irreversible de una filtración de datos sensibles.